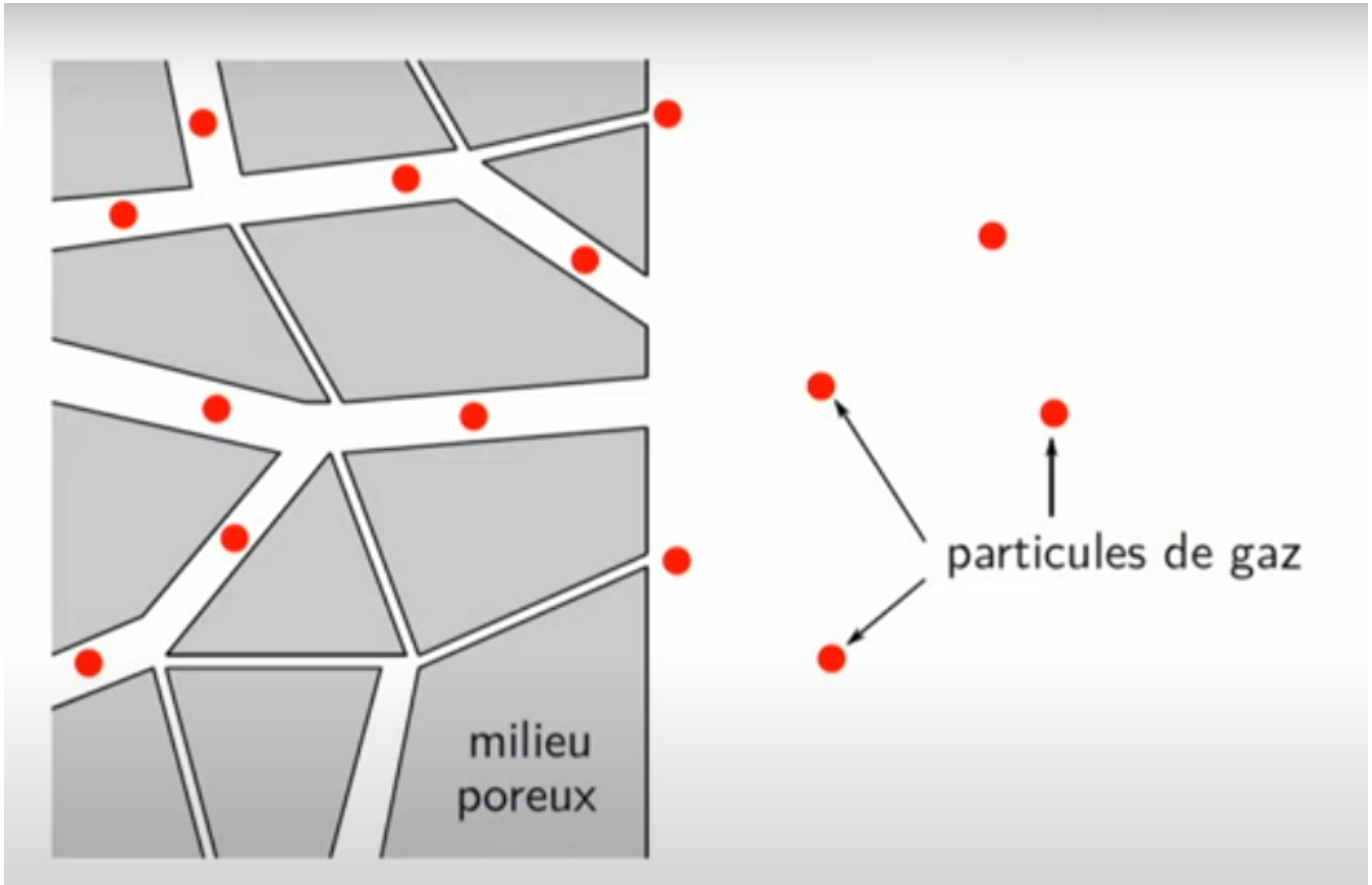
La percolation

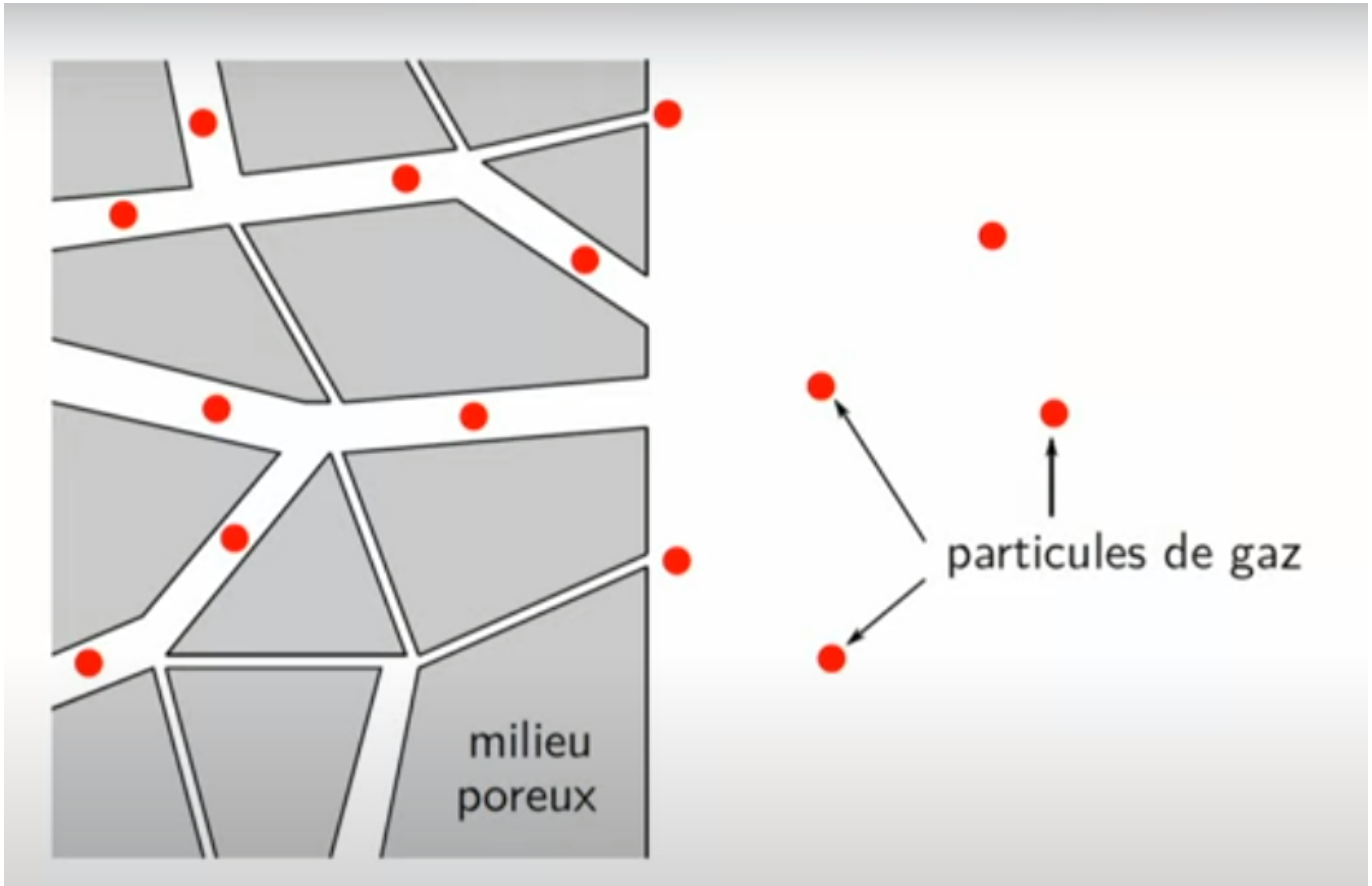
N° SCEI 24790
Yassine Kamal

9 juin 2025

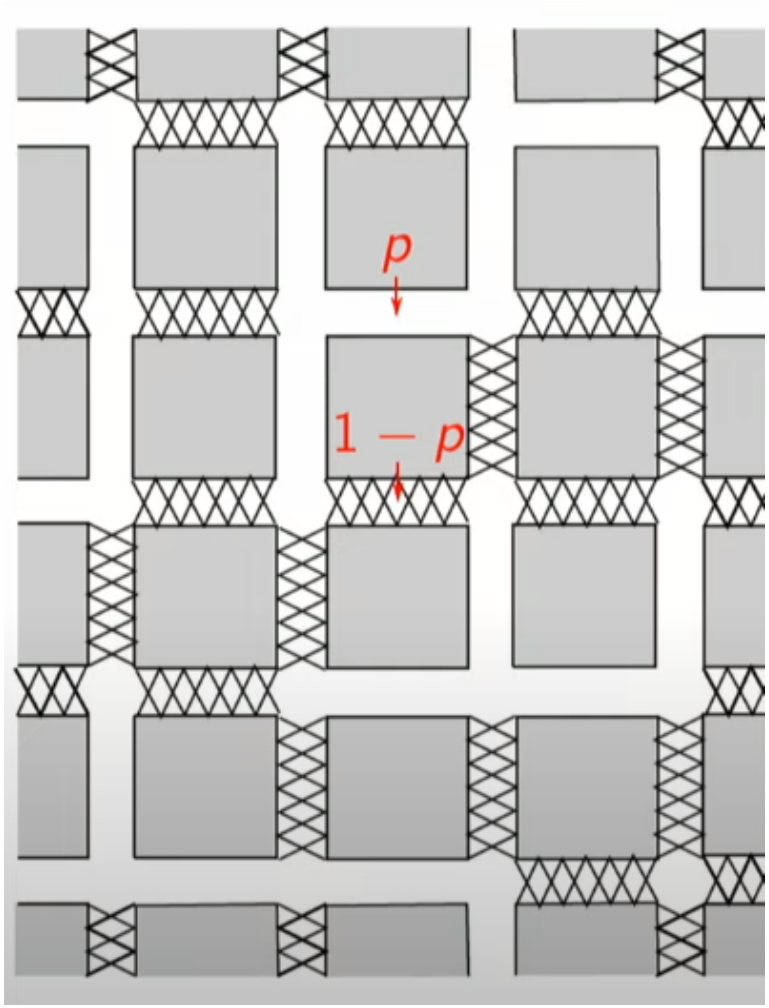
Introduction

un modèle pour les milieux poreux



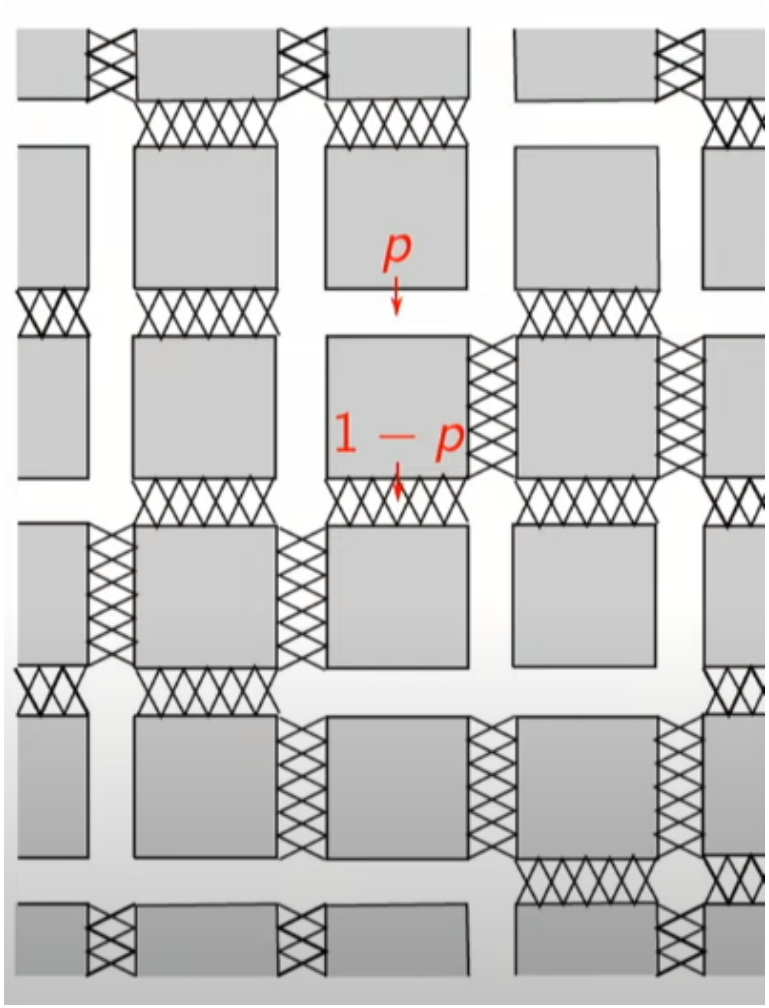


Est ce que les particules pénétrent le milieu en profondeur ?



Simplifications :

- les tuyaux ont une structure régulière ; carré



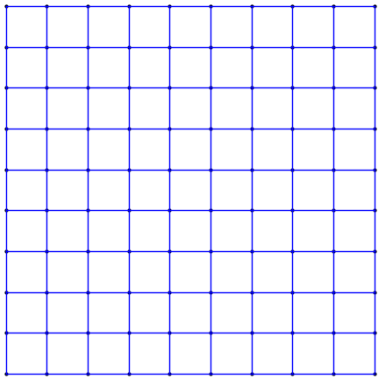
Simplifications :

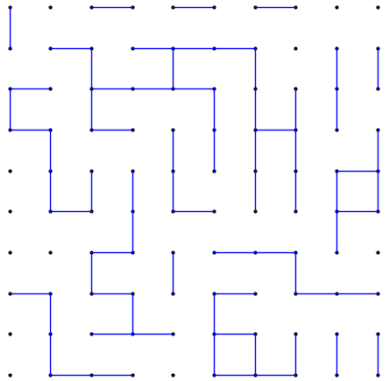
- les tuyaux ont une structure régulière ; carré
- chaque tuyaux est ouvert avec probabilité p et fermé avec probabilité 1-p

Plan

1. Définition
2. mise en évidence de la transition de phase
3. outils et résultats théoriques
4. Algorithme et simulation
5. Percolation dynamique

Dans le graphe $G = (\mathbb{Z}^2, E)$ où
$E = \{\{x, y\} \mid x, y \in \mathbb{Z}^d \text{ et } \|x - y\|_1 = 1\}$ chaque arête de E est gardée avec probabilité p et supprimée avec probabilité $1 - p$.





Percolation p = 0.45

définition : Percolation par arêtes

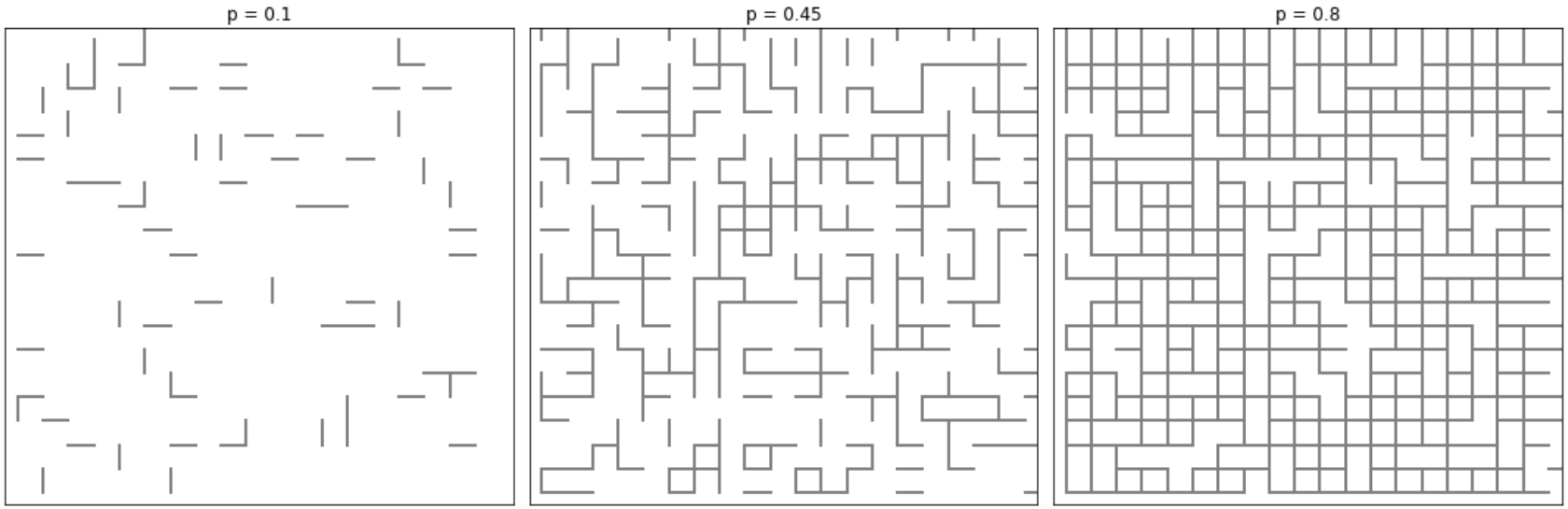
- $(\omega(e))_{e\in E}$ i.i.d. ; $\omega(e) \sim \mathcal{B}(p)$
- $\omega \in \{0,1\}^E$: ω est une configuration ; une arête $e \in E$ est ouverte si $\omega(e) = 1$, et fermée sinon.
- On obtient un sous-graphe de G, $G' = (\mathbb{Z}^2, E_\omega)$, où $E_\omega = \{e \in E \mid \omega(e) = 1\}$.

définition : Percolation par arêtes

- $(\omega(e))_{e\in E}$ i.i.d. ; $\omega(e) \sim \mathcal{B}(p)$
- $\omega \in \{0,1\}^E$: ω est une configuration ; une arête $e \in E$ est ouverte si $\omega(e) = 1$, et fermée sinon.
- On obtient un sous-graphe de G, $G' = (\mathbb{Z}^2, E_\omega)$, où $E_\omega = \{e \in E \mid \omega(e) = 1\}$.
- On dit que x est relié à y, et on note $\{x \longleftrightarrow y\}$ s'il existe un chemin $x = x_0, x_1, \ldots, x_n = y$ tel que $\{x_i, x_{i+1}\} \in E_\omega$ pour tout $i \in \{0, 1, \ldots, n-1\}$.
- On appelle composante ouverte de x : $C_x = \{y \in \mathbb{Z}^2 \mid y \longleftrightarrow x\}$.
- Enfin, on dit que $\{x \longleftrightarrow \infty\}$ si $\text{card}(C_x) = +\infty$.

Percolation d'arêtes

simulation python



- $p = 0.1$
- $p = 0.45$
- $p = 0.8$

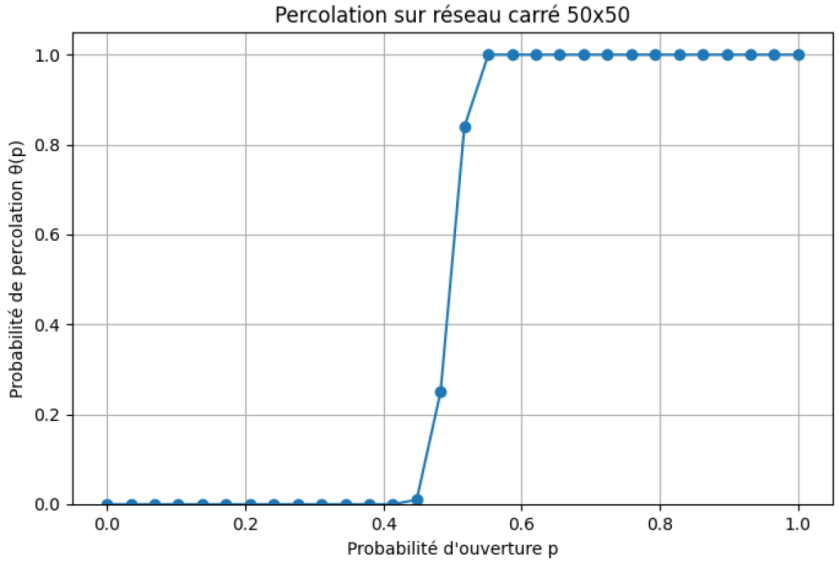
Definition

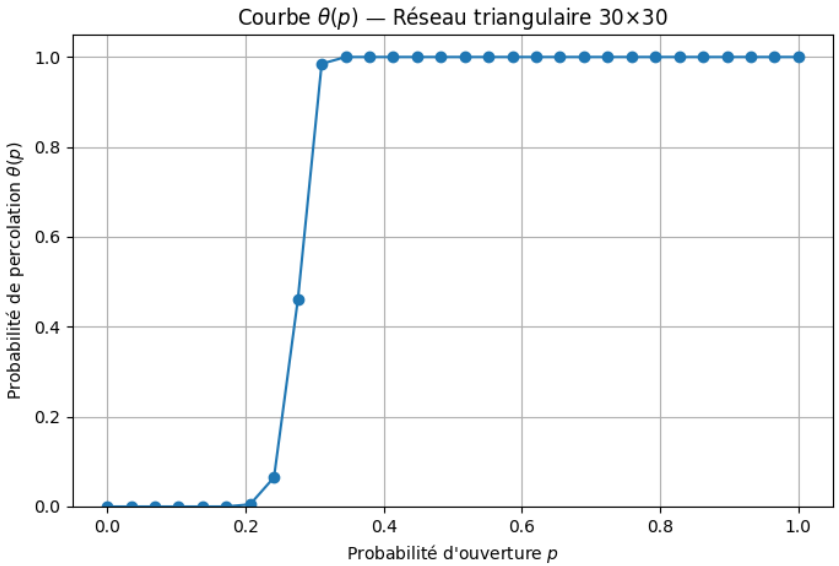
On dit qu'il y a **percolation** lorsque ($\exists x \in E_\omega$ tel que $x \longleftrightarrow \infty$).

$$\theta_x(p) = \mathrm{P}_p(x \longleftrightarrow \infty) = \theta_0(p) = \theta(p)$$

$$\psi(p) = \mathbb{P}_p\left(\bigcup_{x \in E_\omega} \{x \longleftrightarrow \infty\}\right)$$

simulations





Loi du 0-1 de Kolmogorov

Soit A un événement invariant lorsque l'on change l'état d'un nombre fini d'arêtes. Alors $\mathbb{P}_p(A) \in \{0, 1\}$.

Loi du 0-1 de Kolmogorov

Soit A un événement invariant lorsque l'on change l'état d'un nombre fini d'arêtes. Alors $\mathbb{P}_p(A) \in \{0,1\}$.

L'événement $\bigcup_{x \in E_\omega} \{x \longleftrightarrow \infty\}$ est invariant lorsqu'on change l'état d'un nombre fini d'arêtes, donc $\psi(p) \in \{0,1\}$. On pose

$$p_c = \sup\{p \in [0,1] \mid \theta(p) = 0\}$$

Si $p < p_c$, alors $\theta(p) = 0$ par croissance de θ, $\mathbb{Z}^2$ étant dénombrable, on a donc $\psi(p) = 0$.
Si $p > p_c$, alors $\psi(p) \geq \theta(p) > 0$ donc d'après la loi du 0-1, $\psi(p) = 1$.

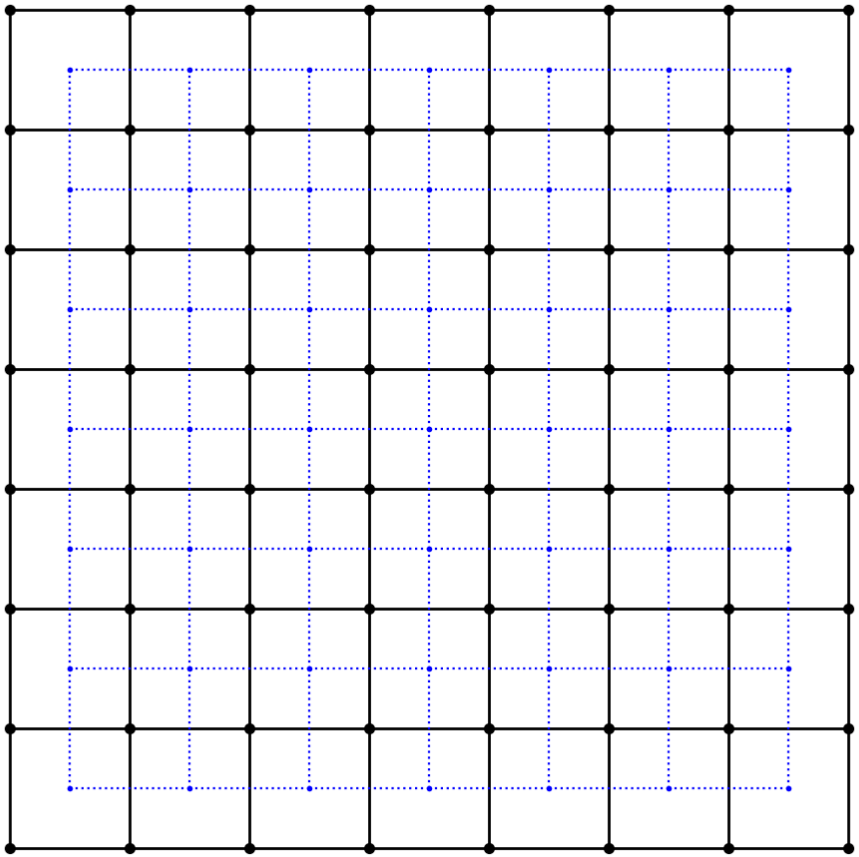
Mise en évidence de la transition de phase

Conséquence

Le modèle de la percolation présente une transition de phase ; On a $\psi(p) = 0$ si $p < p_c$ et $\psi(p) = 1$ si $p > p_c$

- Le graphe dual G^* est le graphe G translaté de $\left(\frac{1}{2}, \frac{1}{2}\right)$.

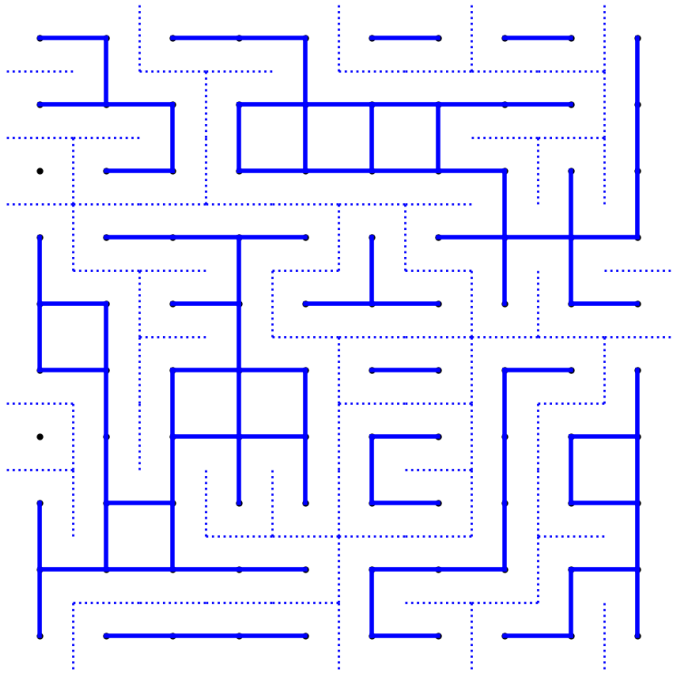
Graphe direct L^2 (noir) et graphe dual L^{2*} (bleu)



Graphe dual : percolation

- On déclare une arête du dual ouverte si elle croise une arête fermée du graphe principal, on la déclare fermée sinon.
- On obtient ainsi une percolation de Bernoulli de paramètre $1 - p$ dans le dual.

Percolation sur $\mathbb{Z}^2$ (noir et bleu), dual (bleu pointillé), $p = 0.5$



Theorem (Inégalité FKG)

Soient X, Y croissantes sur l'espace de percolation :
$\omega \leq \omega' \Rightarrow X(\omega') \geq X(\omega)$. *Alors :*

$$\mathbb{E}[XY] \geq \mathbb{E}[X] \cdot \mathbb{E}[Y]$$

Theorem (Unicité)

Si $p > p_c$, la composante ouverte infinie est presque sûrement unique.

Conséquence

$$p_c(\mathbb{L}^2) = \frac{1}{2}$$

Union-Find : un algorithme pour les graphes

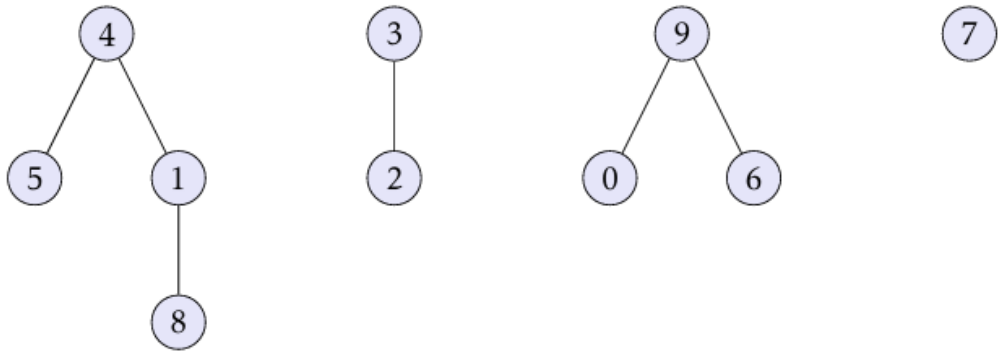


FIGURE 1 – Une représentation de la partition $\{\{1,4,5,8\},\{2,3\},\{0,6,9\},\{7\}\}$.

L'algorithme Union-Find permet :

- la fusion des classes de deux éléments
  (union)
- la recherche du représentant d'un élément
  (find)

Fonctionnement des opérations Union et Find

- **find** : remonter jusqu'à la racine de l'arbre.
- **union** : brancher la racine d'un des deux arbres vers la racine de l'autre.

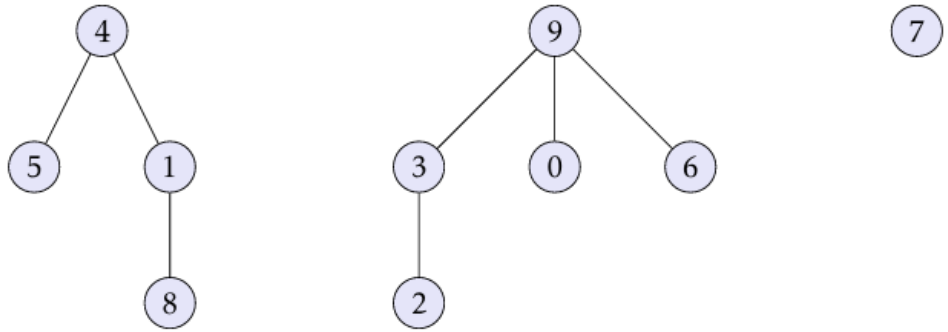


FIGURE 2 – Un exemple d'union des classes $\{2,3\}$ et $\{0,6,9\}$ de la partition précédente.

Optimisation : compression de chemin

- on branche directement sur la racine les différents nœuds rencontrés sur le chemin.

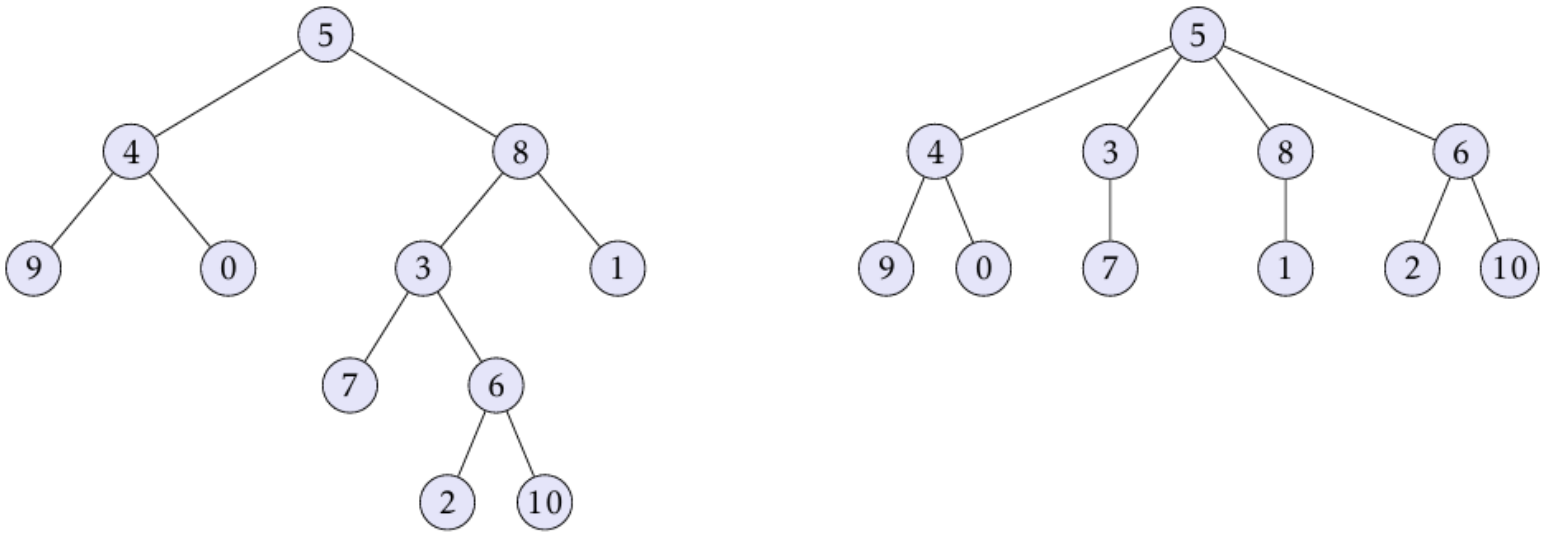


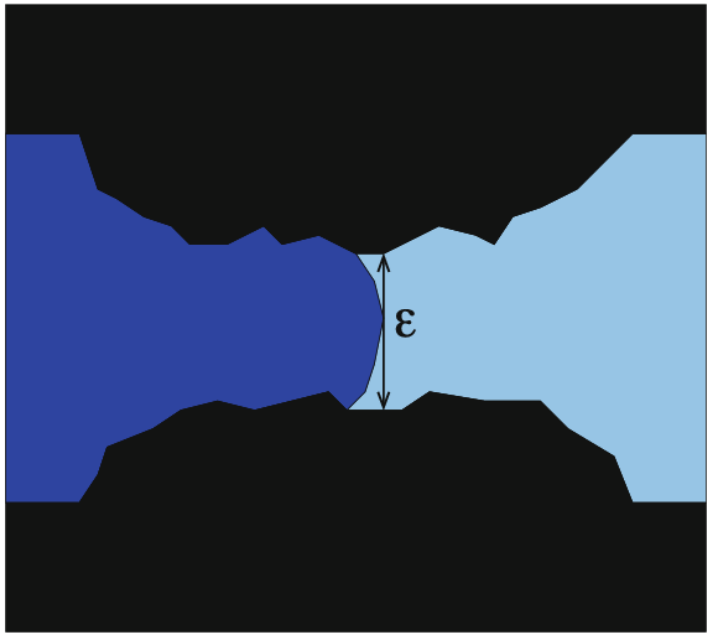
FIGURE 3 – La compression de chemin après avoir recherché le représentant de la classe de 6.

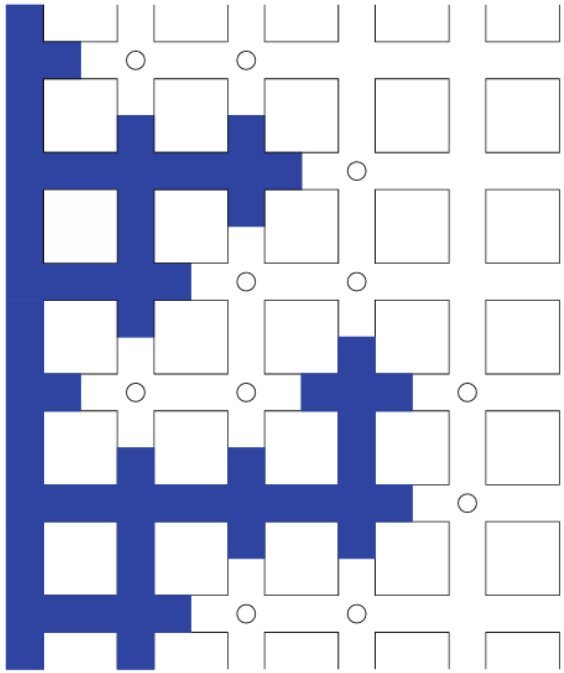
application au calcul des seuils de percolation des réseaux usuels

Réseau	**Seuil estimé p_c**
Carré 2D	0.4946
Triangulaire 2D	0.3426
Hexagonal 2D	0.6207
Cubique 3D	0.2546

Résultats obtenus par simulation numérique (méthode Union-Find, 100 itérations par point).

percolation dynamique :Système physique

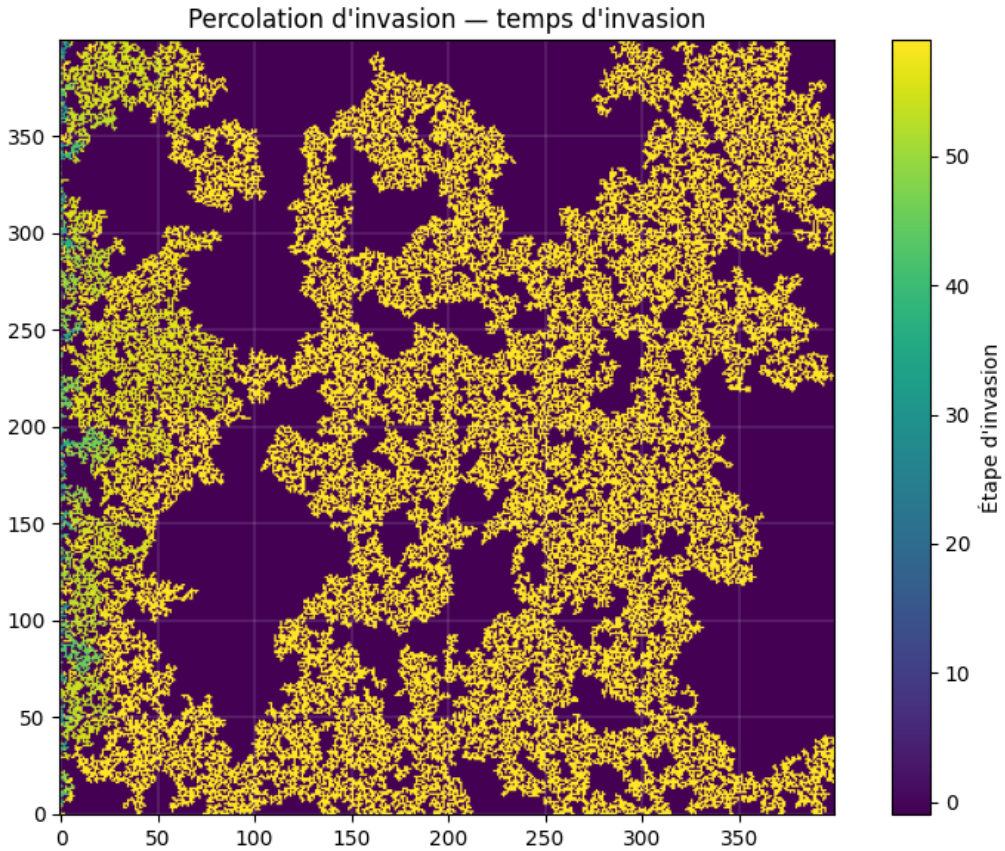




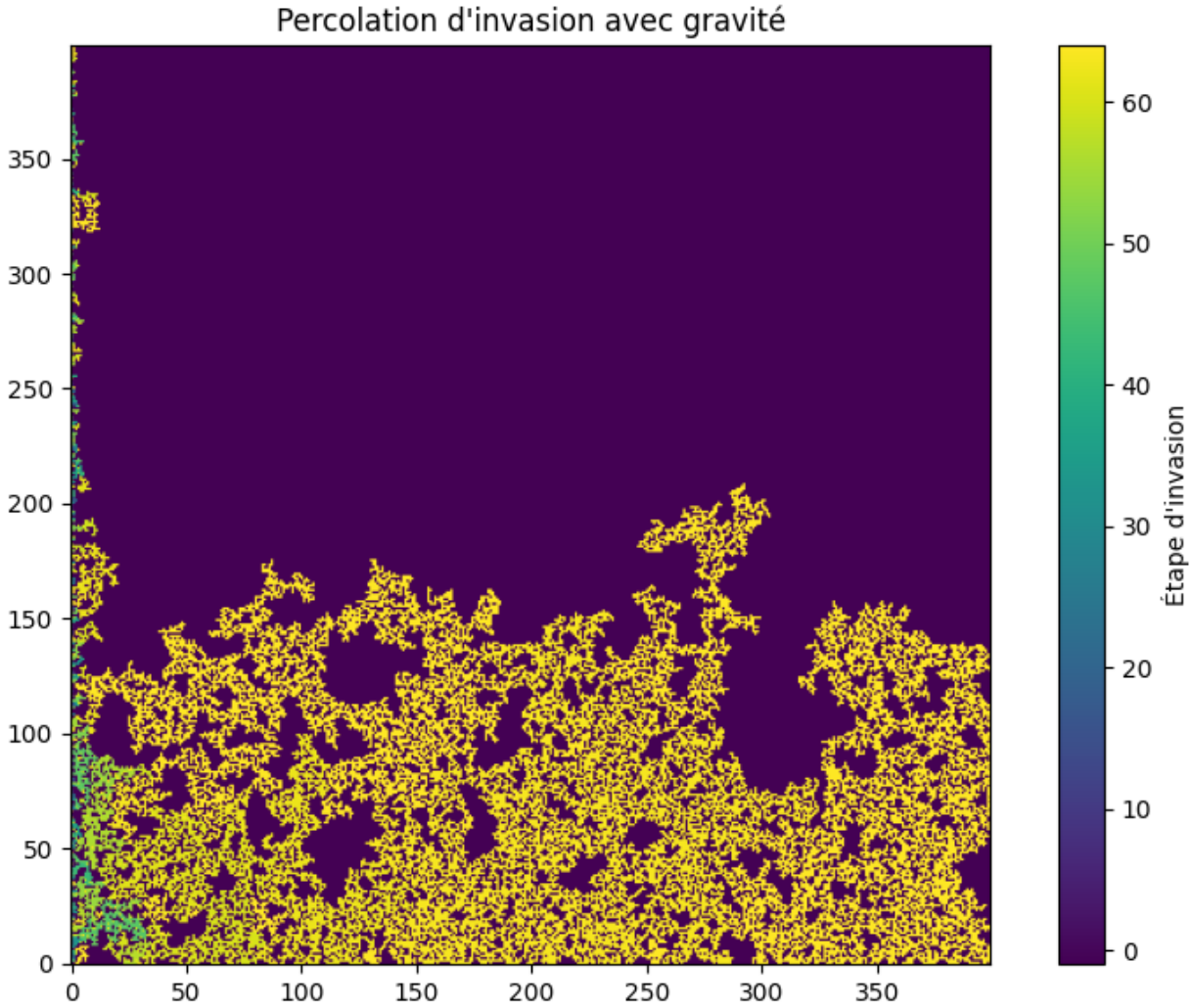
- Milieu poreux saturé
- Injection lente d'un fluide non mouillant
- Seuils d'entrée aléatoires dans chaque site

Modélisation numérique de l'invasion

- Seuils aléatoires $p_{i,j} \in [0, 1]$
- Invasion du site ayant le plus petit seuil
- enregistrement du temps d'invasion



Effet de la gravité sur l'invasion



- Seuil modifié : $p_{i,j} \leftarrow p_{i,j} + g \cdot y$
- Propagation orientée vers le bas

une généralisation : percolation de premier passage

- Temps de passage aléatoires $t(e) > 0$, i.i.d.
- Pour un chemin $\gamma = (e_1, \ldots, e_n)$, on définit :

$$T(\gamma) := \sum_{i=1}^{n} t(e_i)$$

- Temps minimal de u à v :

$$T(u, v) := \inf_{\gamma : u \to v} T(\gamma)$$

- Points mouillés au temps t :

$$B_t := \left\{ x \in \mathbb{Z}^d \mid T(0, x) \leq t \right\}$$

Percolation de premier passage : questions

- Quand un sommet sera-t-il atteint ?
  $T(0, nx) \sim n \cdot \mu(x) \quad$ lorsque $n \to +\infty$
- Comment évolue la zone mouillée ?
  $B_t \sim t \cdot \mathcal{B}_\mu \quad$ lorsque $t \to +\infty$

Fin

Annexe — union find

```python
import numpy as np
import matplotlib.pyplot as plt

def initialiser_union_find(n):
    parent = np.arange(n)
    rang = np.zeros(n, dtype=int)
    return parent, rang

def trouver_racine(parent, x):
    if parent[x] != x:
        parent[x] = trouver_racine(parent, parent[x])
    return parent[x]

def union(parent, rang, x, y):
    rx, ry = trouver_racine(parent, x), trouver_racine(parent, y)
    if rx == ry:
        return
    if rang[rx] < rang[ry]:
        parent[rx] = ry
    else:
        parent[ry] = rx
        if rang[rx] == rang[ry]:
            rang[rx] += 1
```

Annexe — visualisation réseau carré

```python
def creer_reseau_carre(N):
    arretes, positions = [], {}
    for i in range(N):
        for j in range(N):
            idx = i * N + j
            positions[idx] = (j, N - 1 - i)
            if j < N - 1:
                arretes.append((idx, idx + 1))
            if i < N - 1:
                arretes.append((idx, idx + N))
    return arretes, positions, N * N

def visualiser_percolation(N, p):
    arretes, pos, nb = creer_reseau_carre(N)
    parent, rang = initialiser_union_find(nb)
    arretes_ouvertes = [e for e in arretes if np.random.rand() < p]

    for u, v in arretes_ouvertes:
        union(parent, rang, u, v)

    percole = any(
        trouver_racine(parent, g) == trouver_racine(parent, d)
        for g in range(0, N * N, N)
        for d in range(N - 1, N * N, N)
    )

    plt.figure(figsize=(6, 6))
    for u, v in arretes_ouvertes:
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        plt.plot([x0, x1], [y0, y1], 'b-', lw=1)
    for k in pos:
        x, y = pos[k]
        plt.plot(x, y, 'ko', markersize=1.5)
    plt.axis('off')
    plt.title(f"Réseau carré (N={N}, p={p})\n{'Percolation' if percole else 'Pas de percolation'}")
    plt.show()

visualiser_percolation(N=30, p=0.6)
```

Annexe — Courbe $\theta(p)$ — Réseau carré

```python
# Réseau carré : arêtes entre voisins immédiats
def creer_reseau_carre(N):
    arretes = []
    for i in range(N):
        for j in range(N):
            idx = i * N + j
            if j < N - 1:
                arretes.append((idx, idx + 1))     # droite
            if i < N - 1:
                arretes.append((idx, idx + N))     # bas
    return arretes, N * N

# Calcul de θ(p) pour un p donné (moyenne sur plusieurs essais)
def probabilite_percolation_carre(N, p, essais=100):
    succes = 0
    for _ in range(essais):
        arretes, nb_noeuds = creer_reseau_carre(N)
        parent, rang = initialiser_union_find(nb_noeuds)
        arretes_ouvertes = [e for e in arretes if np.random.rand() < p]
        for u, v in arretes_ouvertes:
            union(parent, rang, u, v)
        gauche = [i * N for i in range(N)]
        droite = [i * N + (N - 1) for i in range(N)]
        if any(trouver_racine(parent, g) == trouver_racine(parent, d) for g in gauche for d in droite):
            succes += 1
    return succes / essais

# Tracer la courbe θ(p)
def tracer_theta_carre(N=50):
    ps = np.linspace(0.3, 0.7, 50)
    theta = [probabilite_percolation_carre(N, p) for p in ps]
    plt.plot(ps, theta)
    plt.xlabel("Probabilité p")
    plt.ylabel("Probabilité de percolation θ(p)")
    plt.title(f"θ(p) pour le réseau carré (N = {N})")
    plt.grid()
    plt.show()

# Exécution
tracer_theta_carre()
```

Annexe — Courbe $\theta(p)$ — Réseau triangulaire

```python
# --- Création du réseau triangulaire ---
def create_triangular_lattice(L):
    edges = []
    for i in range(L):
        for j in range(L):
            node = i * L + j
            if j < L - 1:
                edges.append((node, node + 1))        # droite
            if i < L - 1:
                edges.append((node, node + L))        # bas
            if i < L - 1 and j < L - 1:
                edges.append((node, node + L + 1))    # diagonale /
            if i < L - 1 and j > 0:
                edges.append((node, node + L - 1))    # diagonale \
    n_nodes = L * L
    left = [i * L for i in range(L)]
    right = [i * L + (L - 1) for i in range(L)]
    return edges, n_nodes, left, right

# --- Simulation de percolation ---
def percolation_probability(edges, n_nodes, left, right, p, n_iter):
    count = 0
    for _ in range(n_iter):
        parent, rank = uf_init(n_nodes)
        open_edges = [e for e in edges if np.random.rand() < p]
        for u, v in open_edges:
            uf_union(parent, rank, u, v)
        if any(uf_find(parent, l) == uf_find(parent, r) for l in left for r in right):
            count += 1
    return count / n_iter

# --- Tracé de theta(p) ---
def theta_p_triangular(L=30, n_iter=100, resolution=30):
    edges, n_nodes, left, right = create_triangular_lattice(L)
    p_values = np.linspace(0, 1, resolution)
    probs = [percolation_probability(edges, n_nodes, left, right, p, n_iter) for p in p_values]

    plt.figure(figsize=(8, 5))
    plt.plot(p_values, probs, marker='o')
    plt.xlabel("Probabilité d'ouverture p")
    plt.ylabel("Probabilité de percolation θ(p)")
    plt.title(f"Percolation sur réseau triangulaire – L = {L}")
    plt.grid(True)
```

Annexe — temps d'invasion sans gravité

```python
#! C:\Users\USER\OneDrive\Bureau\folder\venv\Scripts\python.exe
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import label  # la version correcte

# ----- Paramètres -----
L = 400  # taille de la grille
p = np.arange(0.0, 0.7, 0.01)  # valeurs de pression croissante
z = np.random.rand(L, L)  # seuils aléatoires
pcluster = np.full((L, L), -1)  # -1 = pas encore envahi
nb_iter = 0  # compteur d'itérations

# ----- Boucle principale -----
for nstep in range(len(p)):
    # sites envahissables à cette pression
    zz = z < p[nstep]
    lw, num = label(zz)

    # détecter les clusters qui touchent le bord gauche
    left = lw[:, 0]
    labels = np.unique(left[left > 0])
    cluster = np.isin(lw, labels)

    # enregistre l'étape d'invasion pour les sites jamais envahis
    newly_invaded = (cluster) & (pcluster == -1)
    pcluster[newly_invaded] = nstep

    nb_iter = nstep + 1

    # test de percolation (gauche à droite)
    right = lw[:, -1]
    span = np.intersect1d(left, right)
    if np.any(span > 0):
        break

# ----- Affichage du temps d'invasion -----
plt.figure(figsize=(8, 6))
plt.imshow(pcluster, origin="lower", cmap="viridis")
plt.colorbar(label="Étape d'invasion")
plt.title("Percolation d'invasion – temps d'invasion")
plt.tight_layout()
plt.show()
```

Annexe — avec gravité

```python
# Paramètres
L = 400
p = np.arange(0.0, 0.7, 0.01)
z = np.random.rand(L, L)

# Ajout de la gravité (gradient vertical)
g = 0.0005  # intensité de la gravité
gradient = g * np.arange(L).reshape((L, 1))  # augmente avec y
z += gradient

# Initialisation
pcluster = np.full((L, L), -1)
invaded = np.zeros((L, L), dtype=bool)
nb_iter = 0

# Boucle principale
for nstep in range(len(p)):
    zz = z < p[nstep]
    lw, num = label(zz)

    left = lw[:, 0]
    labels = np.unique(left[left > 0])
    cluster = np.isin(lw, labels)

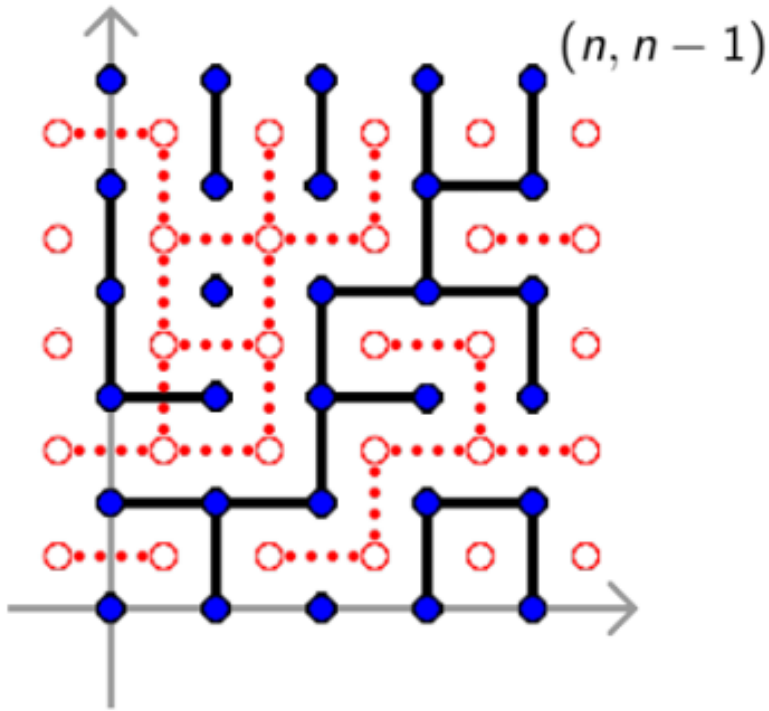
    newly_invaded = cluster & (pcluster == -1)
    pcluster[newly_invaded] = nstep
    invaded |= newly_invaded

    nb_iter = nstep + 1

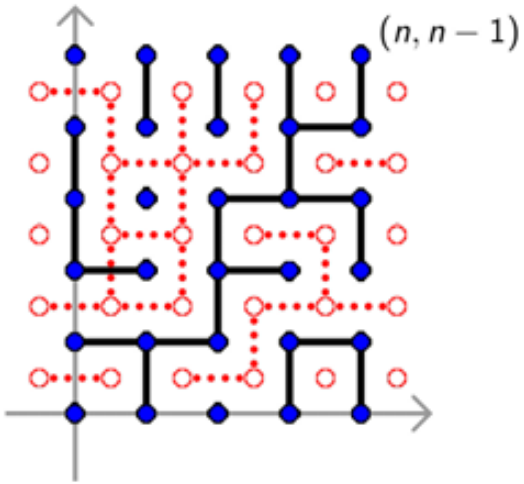
    # Arrêt si percolation
    right = lw[:, -1]
    span = np.intersect1d(left, right)
    if np.any(span > 0):
        break

# Affichage final
plt.figure(figsize=(8, 6))
plt.imshow(pcluster, origin="lower", cmap="viridis")
plt.colorbar(label="Étape d'invasion")
plt.title("Percolation d'invasion avec gravité")
plt.tight_layout()
plt.show()
```

A_n Il existe un chemin ouvert de T_n reliant haut et bas.
A_n^* Il existe un chemin ouvert de T_n^* reliant gauche et droit.



$\{A_n, A_n^*\}$ est un SCE donc
$P_p(A_n) + P_p(A_n^*) = 1$
et $P_p(A_n^*) = P_{1-p}(A_n)$.
Donc $P_{1/2}(A_n) = \frac{1}{2}$

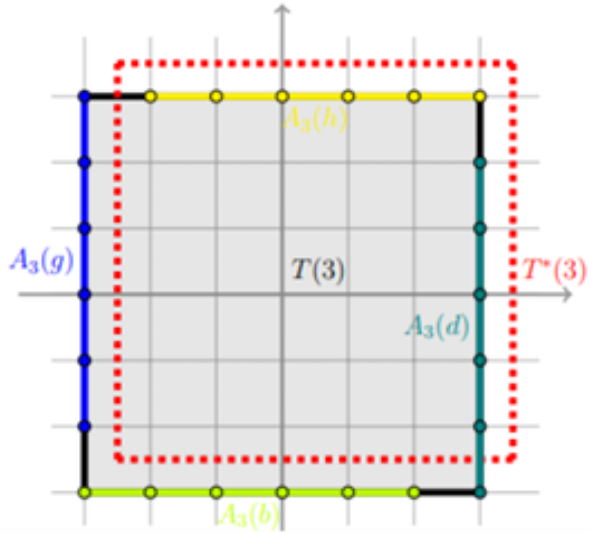


$$A_n \subseteq \bigcup_{x=0}^{n-1} \{|C_{(x,0)}| \geq n\}$$

$$\begin{aligned}
\mathbb{P}_{1/2}(A_n) &\leq \textstyle\sum_{x=0}^{n-1} P_{1/2}(|C_{(x,0)}| \geq n) \\
&\leq \textstyle\sum_{x=0}^{n-1} c_1 e^{-c_2 n} \\
&\leq n c_1 e^{-c_2 n} \underset{n\to\infty}{\longrightarrow} 0
\end{aligned}$$

donc $p_c \leq \frac{1}{2}$

Supposons avoir $p_c < \frac{1}{2}$
$T(n)$ la boîte $(-n,-n),\ldots,(n,n)$.



$A_n(g) = \bigcup_{x=-n-1}^{n} \{(x,-n) \longleftrightarrow \infty\}$ et de même les événements $A_n(d), A_n(h), A_n(b)$ pour les côtés droit, haut et bas de $T(n)$.
$B_n = A_n(g) \cup A_n(d) \cup A_n(h) \cup A_n(b)$

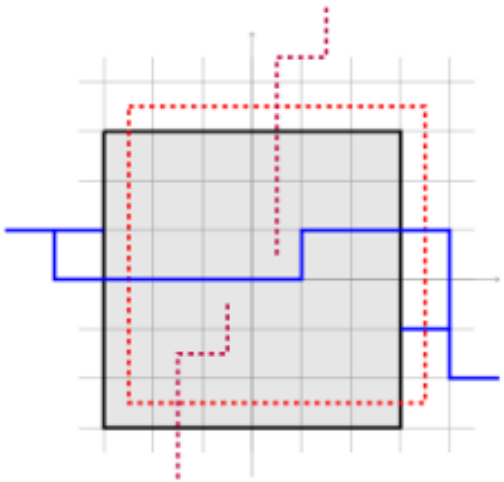
$$\mathbb{P}_{1/2}(B_n) \underset{n\to\infty}{\longrightarrow} \quad \psi\left(\frac{1}{2}\right) = 1$$

$$\mathbb{P}_{1/2}(\overline{B}_n) \leq \mathbb{P}_{1/2}(\overline{A_n(u)})^4 \quad \forall u \in \{g, d, b, h\}$$

donc $\mathbb{P}_{1/2}(A_n(u)) \geq 1 - \sqrt[4]{1 - \mathbb{P}_{1/2}(B_n)}$

$$\text{alors } \mathbb{P}_{1/2}(A_n(u)) \underset{n\to\infty}{\longrightarrow} 1$$

$T^*(n)$ et $A_n^*(u)$ la boîte duale et les événements équivalents dans le dual.



alors $p_c \geq 1/2$

$$C = A_N(g) \cap A_N(d) \cap A_N^*(h) \cap A_N^*(b)$$

d'une part $\mathbb{P}_{1/2}(C) = 0$
et d'autre part $\mathbb{P}_{1/2}(\overline{C}) \leq$
$\mathbb{P}_{1/2}(\overline{A_n(g)}) + \mathbb{P}_{1/2}(\overline{A_n(d)}) +$
$\mathbb{P}_{1/2}(\overline{A_n^*(h)}) + \mathbb{P}_{1/2}(\overline{A_n^*(b)})$